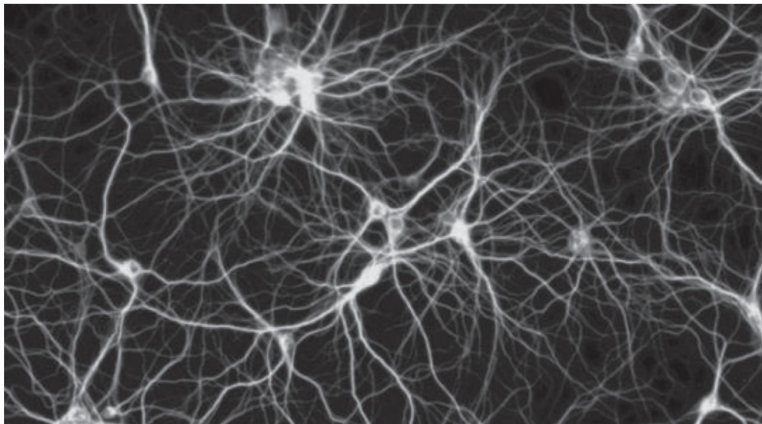


[Chap 11] 인공신경망 기초



인공신경망 개요

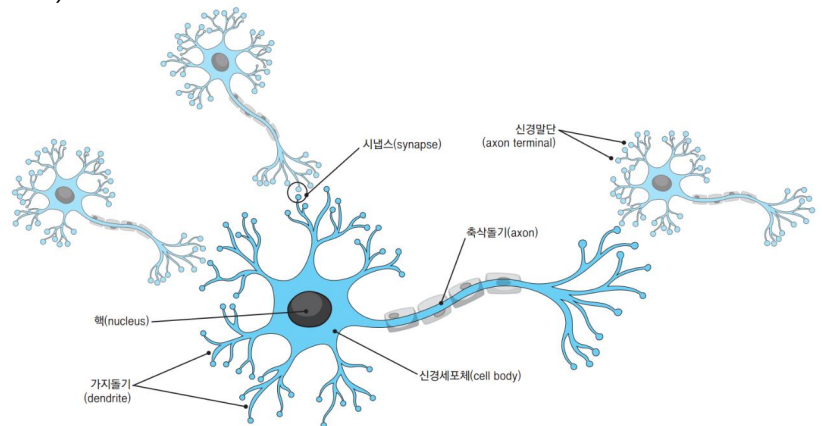
- 인공신경망(Artificial Neural Network, ANN)
 - 줄여서 신경망(Neural Network, NN)이라고도 함
 - 동물의 신경망을 모방하여 지능을 구현하는 모델



[그림 11-2] 신경망의 구조(자료: The Conversation, 2017)³⁾

신경망의 기본단위: 뉴런(neuron)

- 가지돌기(dendrite)로부터 다른 뉴런의 신호를 받아서 축삭돌기(axon)를 통과하여 신경말단(axon terminal)에 연결되어 있는 다른 뉴런들로 신호를 전달
- 뉴런끼리 연결되어 있는 접점을 시냅스(synapse)라고 함
 - 시냅스의 신경전달물질(neurotransmitter)의 농도에 따라 입력 자극에 대한 민감도가 달라짐
 - 자극의 총량이 임계값(threshold)을 넘어서면 다른 뉴런들에게 자극을 전달
- 뉴런들이 그물처럼 연결되어 지능이 발현
 - 인간의 뇌에는 약 1000억 개의 뉴런과 100조 개의 시냅스가 존재



[그림 11-1] 뉴런의 구조

3

뉴런의 기본적 메커니즘

- 입력 자극의 가중합을 임계값(threshold)과 비교

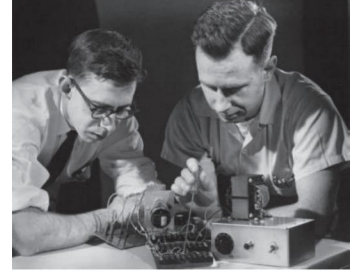
```
1 sum = 0
2 for i in range(0, n): sum += w[i]*x[i]
3 if sum >= threshold : transmit_to_other_neurons()
```

- n: 가지돌기의 입력 시냅스에 연결되어 있는 뉴런의 개수
- x: 입력 시냅스로부터 들어온 모든 입력값
- w: 각 입력 시냅스에 대한 가중치
- 모든 입력 시냅스에 대해 $w[i]$ 와 $x[i]$ 의 가중합을 계산하여 이 값이 특정 임계값 threshold를 넘으면 연결된 다른 뉴런들로 신호를 전파
 - 코드에서는 신호 전파를 `transmit_to_other_neurons()`라는 가상의 함수로 표현
- 임계값을 넘지 않으면 신호는 무시됨

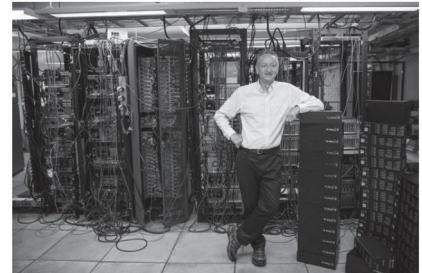
4

인공신경망의 역사

- 퍼셉트론(Perceptron)
 - 1957년 프랭크 로젠블라트(Frank Rosenblatt)가 발표
 - 뉴런 작동을 컴퓨터로 구현, 간단한 분류 수행
- AI의 겨울(AI Winter)
 - 마빈 민스키(Marvin Minsky)는 퍼셉트론의 확장성 부족을 지적
 - 대중과 학계의 관심 소멸
- 역전파 알고리즘(backpropagation algorithm)
 - 제프리 힌튼(Geoffrey Hinton)이 발표
 - 이론적으로 심층신경망의 학습을 가능하게 하였으나, 실제로는 계층 수가 제한적
- 딥 러닝(deep learning)
 - 제프리 힌튼(2006), "A Fast Learning Algorithm for Deep Belief Nets"
 - 심층신경망 학습을 위한 효율적인 방법 제시



[그림 11-3] 퍼셉트론 부품으로 연구중인 프랭크 로젠블라트(왼쪽) (자료: Cornell Chronicle, 2019)

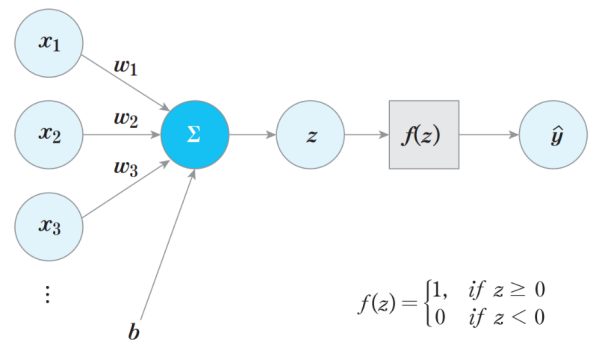


[그림 11-4] 딥러닝의 대부 제프리 힌튼 (자료: U of T news, 2018)

5

퍼셉트론(Perceptron) 모델

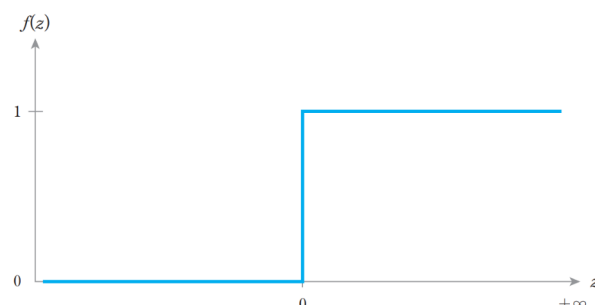
- 최초의 인공신경망 모델
 - 뉴런의 작동방식을 그대로 모방
- 입력값(x_1, x_2, x_3)에 가중치(w_1, w_2, w_3)를 곱하여 가중합 계산
 - $z = w_1x_1 + w_2x_2 + w_3x_3 + \dots + b$
- 가중합이 0 이상이면 1을, 0보다 작으면 0을 출력



[그림 11-5] 퍼셉트론의 구조

$$\hat{y} = f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

- $f(z)$: 활성화함수(activation function)



[그림 11-6] 퍼셉트론의 활성화 함수

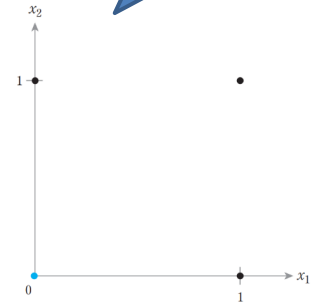
6

퍼셉트론 예제

- 예제 데이터: or 논리연산

〈표 11-1〉 퍼셉트론 예제 데이터

입력값		정답값
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1



정답값 0을 파란색,
1을 검정색 점(●)
으로 표시

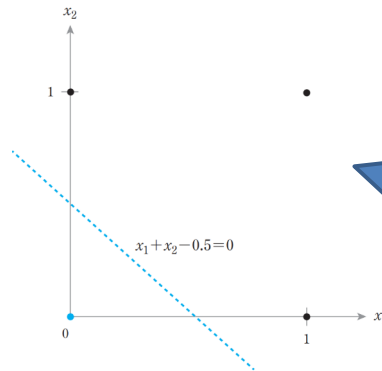
- 목표 분류

$$w_1x_1 + w_2x_2 + b < 0, \text{ if } x_1 = x_2 = 0$$

$$w_1x_1 + w_2x_2 + b \geq 0, \text{ otherwise}$$

- 위의 식을 만족하는 값의 예

$$- w_1 = 1, w_2 = 1, b = -0.5$$



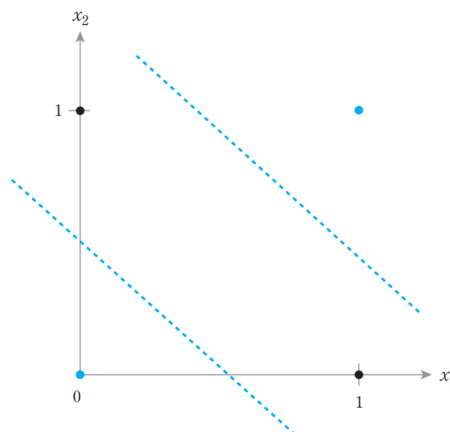
퍼셉트론은
선형 분류
(linear classification)
를 수행

〔그림 11-8〕 퍼셉트론 파라미터의 설정

7

단일 퍼셉트론으로 불가능한 경우의 예

- xor 논리연산 예제
 - $x_1 = x_2$ 이면 0, $x_1 \neq x_2$ 이면 1로 예측해야 함
 - 최소 두 개의 직선이 있어야 분류 가능
- 단일 퍼셉트론으로는 분류 불가능
 - 퍼셉트론을 두 계층으로 구성하면 분류 가능하나, 학습 방법이 없음

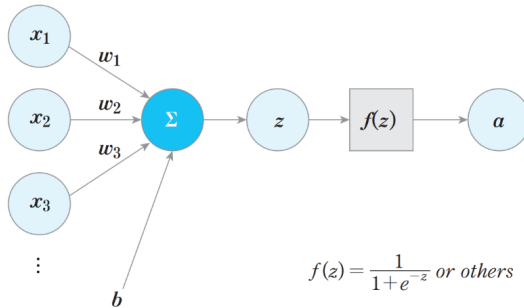


〔그림 11-9〕 한 개의 퍼셉트론으로 해결할 수 없는 분류의 예

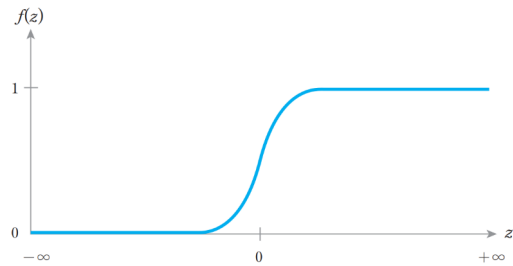
8

현대적인 뉴런 모델

- 활성화 함수를 개선하여 다계층 신경망의 학습을 가능하게 함
 - 퍼셉트론과 구조는 동일하나, 시그모이드 함수를 활성화 함수로 사용
 - z 의 값에 관계없이 $0 < f(z) < 1$ 이 됨



[그림 11-10] 현대적인 뉴런의 구조



[그림 11-11] 시그모이드 활성화 함수

- 예측값 계산 과정
 - 가중합 z 를 계산
 - z 에 활성화함수를 적용하여 a 를 계산
 - a 의 값이 0.5 이상이면 예측값을 1로, 아니면 0으로 출력

$$z = w_1x_1 + w_2x_2 + w_3x_3 + \dots + b$$

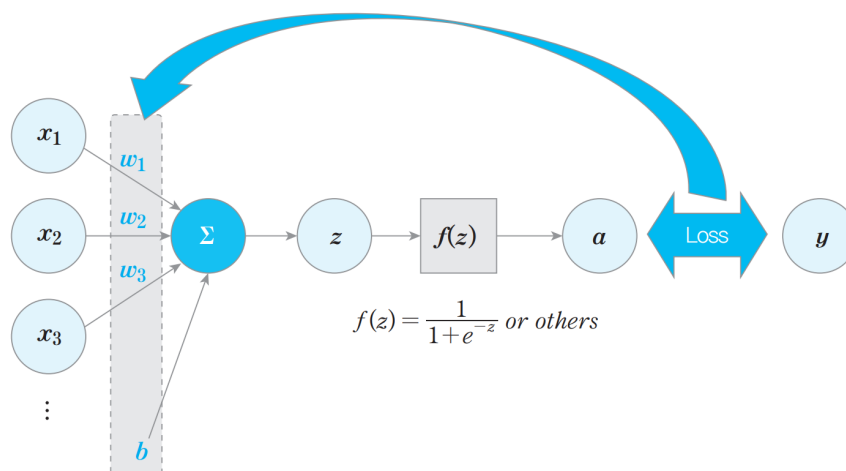
$$a = f(z) = \frac{1}{1+e^{-z}}$$

$$\hat{y} = \begin{cases} 1, & \text{if } a \geq 0.5 \\ 0, & \text{if } a < 0.5 \end{cases}$$

9

단일 뉴런에서의 오차 역전파

- 오차 역전파(error backpropagation)의 개념
 - 정답값과 예측값의 오차를 통해 손실함수(loss function) 계산
 - 손실함수를 줄이는 방향으로 파라미터($w_1, w_2, \dots, b$)를 조정



[그림 11-12] 오차 역전파 과정

단일 뉴런에서의 오차 역전파 (계속)

- 손실함수는 엔트로피 활용

$$L(w_1, w_2, w_3, \dots, b) = -\frac{1}{n} \sum_{i=1}^n (y_i \log(a_i) + (1 - y_i) \log(1 - a_i))$$

- 손실함수의 그래디언트를 구하여 파라미터 수정에 사용 (연쇄미분)
 - 그래디언트는 연쇄미분을 통해 계산하며, 최종적으로 단순한 식으로 도출됨

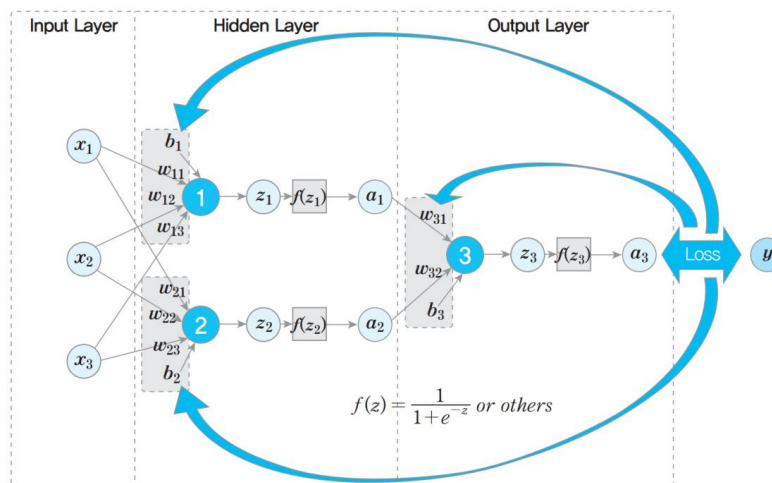
$$\bullet \frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (a_i - y_i) x_{i,j}$$

$$\bullet \frac{\partial L}{\partial b} = \frac{\partial L}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial b} = \frac{1}{n} \sum_{i=1}^n (a_i - y_i)$$

11

다계층 인공신경망과 역전파 알고리즘

- 계층의 구분
 - 입력층(input layer): 입력값 x_1, x_2, x_3 이 있는 계층
 - 출력층(output layer): 출력 뉴런(3번 뉴런)이 있는 계층
 - 은닉층(hidden layer): 입력, 출력을 제외한 계층(1, 2번 뉴런)
- 계층 개수를 셀 때는 입력층은 제외하므로, 아래 그림은 2계층 신경망임



[그림 11-13] 다계층 신경망의 오차 역전파

12

다계층 인공신경망과 역전파 알고리즘 (계속)

- 예측값의 계산
 - 1, 2번 뉴런에서 각각 예측값 a_1, a_2 를 계산

$$a_1 = \frac{1}{1+e^{-z_1}}, \text{ 여기서 } z_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1$$

$$a_2 = \frac{1}{1+e^{-z_2}}, \text{ 여기서 } z_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2$$

- 3번 뉴런은 a_1, a_2 를 입력값으로 하여 a_3 를 계산

$$a_3 = \frac{1}{1+e^{-z_3}}, \text{ 여기서 } z_3 = w_{31}a_1 + w_{32}a_2 + b_3$$

- 최종 출력값 y 는 a_3 가 0.5 이상이면 1, 아니면 0으로 결정

- 손실함수
 - 단일 뉴런과 마찬가지로 엔트로피 함수 활용

$$L = -\frac{1}{n} \sum_{i=1}^n \{y(i) \log(a_3(i)) + (1-y(i)) \log(1-a_3(i))\}$$

13

다계층 인공신경망과 역전파 알고리즘 (계속)

- 역전파 알고리즘(backpropagation algorithm)
 - 다계층 인공신경망에서 그래디언트를 계산하기 위한 방법
 - 출력층(3번 뉴런)의 그래디언트는 단일 뉴런과 동일한 방법으로 계산됨
 - $\frac{\partial L}{\partial w_{31}} = \frac{\partial L}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_{31}} = \frac{1}{n} \sum_{i=1}^n (a_3(i) - y(i)) a_1(i)$
 - $\frac{\partial L}{\partial w_{32}} = \frac{\partial L}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_{32}} = \frac{1}{n} \sum_{i=1}^n (a_3(i) - y(i)) a_2(i)$
 - $\frac{\partial L}{\partial w_{31}} = \frac{\partial L}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial b} = \frac{1}{n} \sum_{i=1}^n (a_3(i) - y(i))$
 - 은닉층(2번 뉴런)의 그래디언트는 연쇄미분이 더 길어지지만, 최종 계산결과는 단순한 식의 곱으로 나타남

$$\frac{\partial L}{\partial w_{1j}} = \frac{\partial L}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{1j}} = \frac{1}{n} \sum_{i=1}^n (a_3(i) - y(i)) w_{31} a_1(i) (1 - a_1(i)) x_j(i)$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial b_1} = \frac{1}{n} \sum_{i=1}^n (a_3(i) - y(i)) w_{31} a_1(i) (1 - a_1(i))$$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_{21}} = \frac{1}{n} \sum_{i=1}^n (a_3(i) - y(i)) w_{32} a_2(i) (1 - a_2(i)) x_j(i)$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial b_2} = \frac{1}{n} \sum_{i=1}^n (a_3(i) - y(i)) w_{32} a_2(i) (1 - a_2(i))$$

14

역전파 예제 데이터

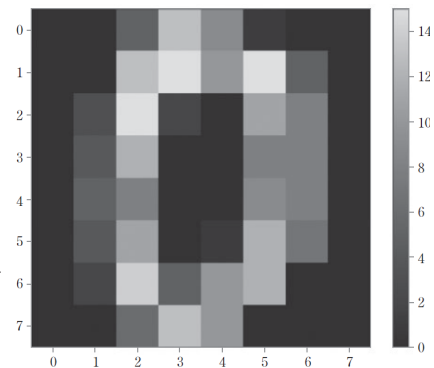
- 데이터 읽어오기: 저해상도(8x8)의 손글씨 데이터

```
1 from sklearn import datasets
2 dataset = datasets.load_digits()
3 x_data = dataset.data
4 y_data = dataset.target
5
6 print('X shape = ', x_data.shape)
7 print('Y shape = ', y_data.shape)
```

X shape = (1797, 64)
Y shape = (1797,)

- 데이터의 시각화: 0번 데이터의 예

```
1 print('First x data = ', x_data[0])
2 print('First y data = ', y_data[0])
3
4 import matplotlib.pyplot as plt
5 plt.imshow(x_data[0].reshape(8,8))
6 plt.colorbar()
7 plt.show()
```



15

데이터 전처리

- 데이터 가공

```
1 import numpy as np
2 y_data = (y_data % 2).astype(np.float32)
3 x_data = (x_data / 15.0).astype(np.float32)
```

- (2행) 정답을 짝수와 홀수만 구분하도록 가공
 - 이렇게 하면 출력 뉴런은 하나만 두면 됨
- (3행) 입력값은 0과 1 사이가 되도록 가공

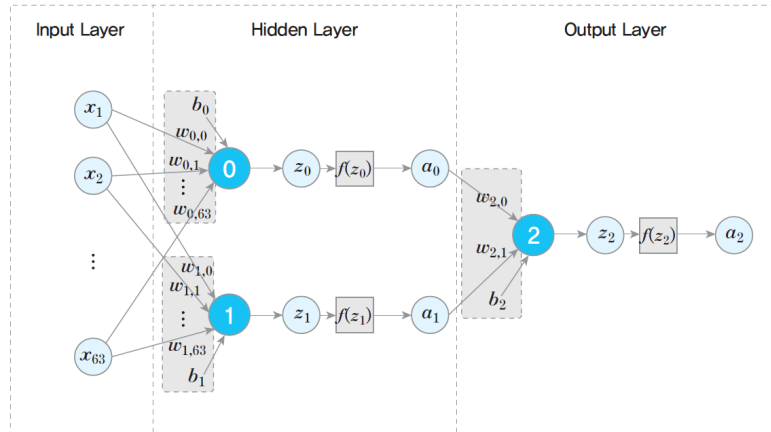
- 학습 데이터와 테스트 데이터로 분할

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data, test_size=0.2)
```

- 20%를 테스트 데이터로 사용

역전파의 구현

- 구현하려는 신경망의 구조
 - 출력층 뉴런: 2번
 - 은닉층 뉴런: 0, 1번
 - 입력층: 64개의 픽셀 밝기값



[그림 11-16] 구현하려는 인공신경망 구조

- 파라미터를 모두 0으로 초기화

```
1 w0 = np.zeros(64)
2 b0 = 0
3 w1 = np.zeros(64)
4 b1 = 0
5 w2 = np.zeros(2)
6 b2 = 0
```

17

역전파의 구현 (계속)

- 뉴런 하나의 출력을 계산하는 함수
 - (3~4행) 가중합 z 계산
 - (5행) 활성화 함수

```
1 def output(data, w, b):
2     z = b
3     for j in range(len(data)):
4         z += w[j]*data[j]
5     a = 1/(1+np.exp(-z))
6     return a
```

- 전체 신경망의 출력값 계산 함수
 - (6~7행) 은닉층 계산
 - (8행) 출력층 계산

```
1 def predict(x, w0, b0, w1, b1, w2, b2):
2     n = len(x)
3     a = np.zeros(n)
4     for i in range(n):
5         a_hid = np.zeros(2)
6         a_hid[0] = output(x[i], w0, b0)
7         a_hid[1] = output(x[i], w1, b1)
8         a_out = output(a_hid, w2, b2)
9         a[i] = a_out
10    return a
```

- 손실함수: 엔트로피 함수 사용

```
1 def loss(a, y):
2     l = y*np.log(a) + (1-y)*(np.log(1-a))
3     return -np.mean(l)
```

18

역전파의 구현 (계속)

- 그래디언트 계산

- (21~22, 36행) $\frac{\partial L}{\partial w_{2,j}} = \frac{1}{n} \sum_{i=0}^{n-1} (a_2(i) - y(i)) a_j(i)$ 계산
- (23, 37행) $\frac{\partial L}{\partial b_2} = \frac{1}{n} \sum_{i=0}^{n-1} (a_2(i) - y(i))$ 계산
- (25~27, 32~33행) w_0, b_0 에 대한 그래디언트 계산
$$\frac{\partial L}{\partial w_{0,j}} = \frac{1}{n} \sum_{i=0}^{n-1} (a_2(i) - y(i)) w_{2,0} (a_0(i) (1 - a_0(i))) x_j(i)$$
$$\frac{\partial L}{\partial b_0} = \frac{1}{n} \sum_{i=0}^{n-1} (a_2(i) - y(i)) w_{2,0} (a_0(i) (1 - a_0(i)))$$
- (29~31, 34~35행) w_1, b_1 에 대한 그래디언트 계산
$$\frac{\partial L}{\partial w_{1,j}} = \frac{1}{n} \sum_{i=0}^{n-1} (a_2(i) - y(i)) w_{2,1} (a_1(i) (1 - a_1(i))) x_j(i)$$
$$\frac{\partial L}{\partial b_1} = \frac{1}{n} \sum_{i=0}^{n-1} (a_2(i) - y(i)) w_{2,1} (a_1(i) (1 - a_1(i)))$$

```
12 for i in range(n):
13     x = x_train[i]
14     y = y_train[i]
15
16     a_hid = np.zeros(2)
17     a_hid[0] = output(x, w0, b0)
18     a_hid[1] = output(x, w1, b1)
19     a_out = output(a_hid, w2, b2)
20
21     for j in range(2):
22         dw2[j] += (a_out - y) * a_hid[j]
23     db2 += (a_out - y)
24
25     for j in range(64):
26         dw0[j] += (a_out - y) * w2[0] * a_hid[0] * (1 - a_hid[0]) * x[j]
27         db0 += (a_out - y) * w2[0] * a_hid[0] * (1 - a_hid[0])
28
29     for j in range(64):
30         dw1[j] += (a_out - y) * w2[1] * a_hid[1] * (1 - a_hid[1]) * x[j]
31         db1 += (a_out - y) * w2[1] * a_hid[1] * (1 - a_hid[1])
32
33     dw0 /= n
34     db0 /= n
35     dw1 /= n
36     db1 /= n
37     dw2 /= n
38     db2 /= n
```

19

역전파의 구현 (계속)

- 역전파 부분 전체 코드

```
1 eta = 0.1
2 for epoch in range(2000):
3     a = predict(x_train, w0, b0, w1, b1, w2, b2)
4     print('epoch', epoch, 'loss = ', loss(a, y_train))
5
6     dw0 = np.zeros(64)
7     db0 = 0
8     dw1 = np.zeros(64)
9     db1 = 0
10    dw2 = np.zeros(2)
11    db2 = 0
12    for i in range(n):
13        x = x_train[i]
14        y = y_train[i]
15
16        a_hid = np.zeros(2)
17        a_hid[0] = output(x, w0, b0)
18        a_hid[1] = output(x, w1, b1)
19        a_out = output(a_hid, w2, b2)
20
21        for j in range(2):
22            dw2[j] += (a_out - y) * a_hid[j]
23        db2 += (a_out - y)
```

```
25        for j in range(64):
26            dw0[j] += (a_out - y) * w2[0] * a_hid[0] * (1 - a_hid[0]) * x[j]
27            db0 += (a_out - y) * w2[0] * a_hid[0] * (1 - a_hid[0])
28
29        for j in range(64):
30            dw1[j] += (a_out - y) * w2[1] * a_hid[1] * (1 - a_hid[1]) * x[j]
31            db1 += (a_out - y) * w2[1] * a_hid[1] * (1 - a_hid[1])
32
33        dw0 /= n
34        db0 /= n
35        dw1 /= n
36        db1 /= n
37        dw2 /= n
38        db2 /= n
39
40        w0 -= eta * dw0
41        b0 -= eta * db0
42        w1 -= eta * dw1
43        b1 -= eta * db1
44        w2 -= eta * dw2
45        b2 -= eta * db2
```

실행 코드 전체는 ex11-4.py 참고

예측 및 성능 지표

- (94~98행) 예측 수행
 - 출력값 a 가 0.5 이상이면 1, 아니면 0으로 예측
- (100~103행) 성능지표 출력
 - 정확도 93%로 양호한 예측성능 확인

```
94 predictions = predict(x_test, w0,b0,w1,b1,w2,b2)
95 predLabels=[]
96 for y in predictions:
97     if y<0.5: predLabels.append(0)
98     else: predLabels.append(1)
99
100 from sklearn.metrics import confusion_matrix
101 from sklearn.metrics import classification_report
102 print(confusion_matrix(y_test, predLabels))
103 print(classification_report(y_test, predLabels))
```

```
[[171 13]
 [ 14 162]]
```

	precision	recall	f1-score	support
0.0	0.92	0.93	0.93	184
1.0	0.93	0.92	0.92	176
accuracy			0.93	360
macro avg	0.93	0.92	0.92	360
weighted avg	0.93	0.93	0.92	360

실행 코드 전체는 ex11-5.py 참고